

REPORTE TAREA 1

ALGORITMOS Y COMPLEJIDAD

«Más allá de la notación asintótica: Análisis experimental de algoritmos de ordenamiento y multiplicación de matrices.»

Felipe Gajardo

11 de septiembre de 2026

04:10

1. Introducción

En esta tarea se estudiara como actuan ciertos algoritmos que resuelven los problemas computacionales de ordenar una lista de arreglos, que en este caso sera numerica, y la multiplicacion de matrices. En base a estos problemas se compararan distintos algoritmos para ver como estos actuan frente a distintos tipos de entradas segun el problema que trbajen , el tiempo de ejecucion servira para ver cual es mas eficiente y el uso de memoria por igual. Estos son los factores principales para determinar que algoritmos son mejores en ciertos casos, tambien comprobaremos si lo esperado teoricamente se cumple con las pruebas.

2. Entorno experimental

Para mostrar como es nuestro enntorno donde haramos las pruebas de estos algoritmos, todo se ejecuto en un sistema operativo windous mas especifico en esta version: Microsoft Windows 11 Home Single Language 10.0.2620 , con un procesador 12th Gen Intel(R) Core(TM) i5-12450HX, ademas de una memoria ram de 16 [Gb]. Para las pruebas se usaron dos lenguajes de programacion

Python y C++, en las versiones 3.13.0 y 13.2.0 respectivamente.

3. Resultados y Análisis

En la fabricación de los gráficos para describir los comportamientos de estos algoritmos, se prefirió usar gráficos que comparen memoria o tiempo contra el tamaño del arreglo o matriz, según su característica, además de su dominio de valores de los elementos.

Para respetar el tamaño que se solicita en esta parte, solo mostraré un gráfico si se ve una anomalía respecto a lo que se espera teóricamente. Pero primero, describiré las pruebas que se hicieron a las matrices.

En primera instancia, por la complejidad del algoritmo de Strassen que es $O(n^{\log_2 7})$, en las mediciones de tiempo vs. tamaño de la entrada, se mostró que el algoritmo naive fue mucho más rápido. Pero esto es consecuencia de que, como uso un promedio de los casos a, b y c en los distintos tamaños, pierdo esa variación; porque viendo las mediciones como tal en el archivo, sí hay diferencia a favor de Strassen.

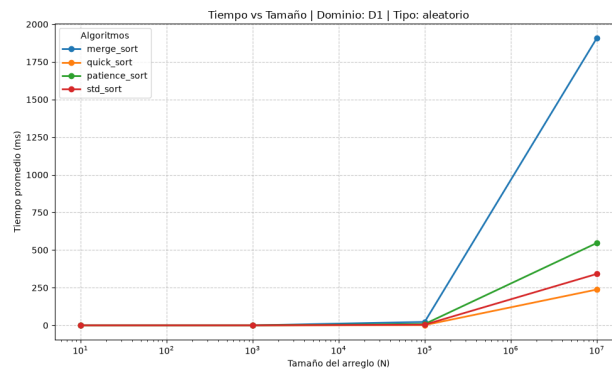
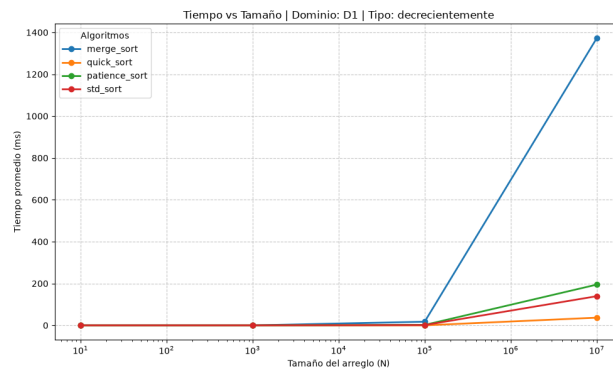


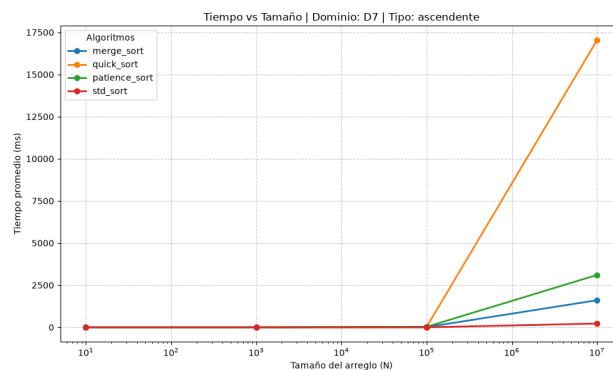
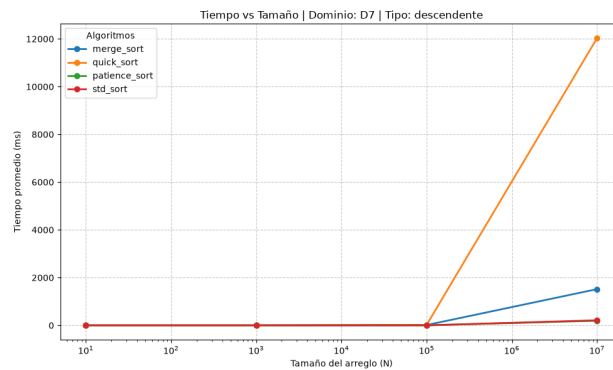
Figura 1: Enter Caption

En cuanto a la memoria, era de esperar que Strassen use más por la separación en pequeñas matrices, haciendo más uso del árbol de recursión. En cambio, con el naive, como son operaciones elementales, su coste de memoria no es tan alto. También se vio que, entre los tipos de matrices que se usaron de entrada, los resultados no variaron mucho en su comportamiento.

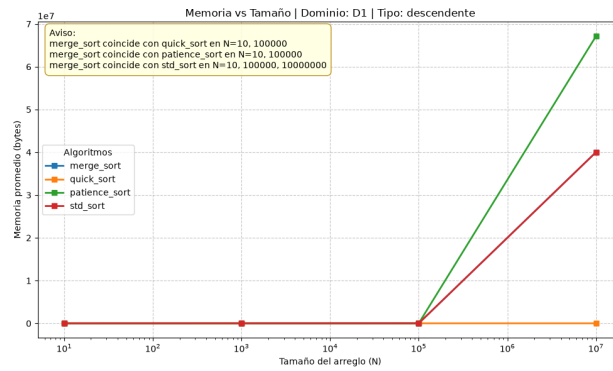
Ahora bien, en las pruebas hechas en los arreglos, si hablamos de tiempo, se hicieron varias observaciones. Primero, en cuanto a cómo se comportaron los algoritmos según el rango donde se mueven los valores de los elementos de los arreglos, los que forman parte de D1 siguen una misma jerarquía que no varía mucho: en último lugar está el merge sort, luego viene patience sort, después std sort y, por último, quick sort (estos dos últimos por poca diferencia). Pero, en la mayoría de los tipos de arreglos de este dominio, el quick sort se ve como el más eficiente si medimos su tiempo de ejecución.



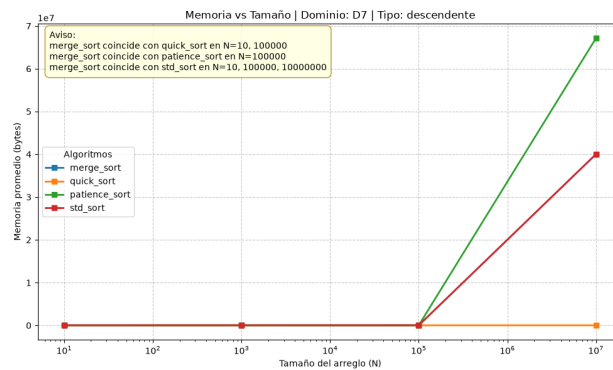
Ahora, si vemos los dominios con valores de elementos mucho más grandes, el comportamiento anterior cambia. El std sort sigue siendo el más eficiente en la mayoría de los casos, pero ahora los tipos de arreglos hacen que ciertos algoritmos se destaquen más, como el patience sort en arreglos descendentes, aunque en los aleatorios no rinde muy bien. También se comprueba el comportamiento del quick sort en los peores casos, cuando el arreglo es ascendente y descendente.



Continuando con la perspectiva de la memoria en el dominio D1, el comportamiento del std sort y merge sort es casi idéntico, manteniéndose en un uso de memoria intermedio en comparación con los otros. Aunque en algunos tipos de arreglos coinciden en el uso de memoria con el patience sort (cuando es descendente), el patience sort eleva su uso de memoria en la mayoría de los casos.



Siguiendo con el análisis de la memoria respecto al tamaño, acá en el dominio D7 la jerarquía de quién usa más o menos memoria se mantiene igual independiente del tipo de arreglo. El que más usa es el patience sort; el merge sort y el std sort tienen un comportamiento similar, y por último, el quick sort sería el que menos memoria usó en su ejecución.



4. Conclusiones

En conclusión, comparándolo con el naive, si bien en promedio los algoritmos Strassen no se vio como un algoritmo rápido, sí se vio que en casos puntuales es mucho más rápido, independiente del tipo de matriz; pero naive fue mucho más eficiente en memoria. En cuanto a los algoritmos de arreglos, se vio la superioridad del sort que forma parte de la librería estándar de C++; solo se vio sobrepasado por otros algoritmos en pruebas muy específicas, pero se notó que para la mayoría de casos es la mejor opción.